Why Did We?: Project Documentation (Free LLM Edition)

Hackathon: Graph Hacks: Context for AI Agents (WeMakeDevs, powered by FalkorDB)
Primary submission: Track 01, Agents That Act on Connected Data Also supports: Track 02, Agent Memory and Coordination; Track 03, Company Brain LLM providers: Ollama (local, free) or Groq (hosted, free tier), selected by configuration

---

1. Overview

1.1 The problem

Teams make decisions in meetings, tickets, and chats, then lose the reasoning weeks later. When someone asks "why did we drop Redis for the cache?", the answer is scattered across messages and people's memories. When an assumption changes (a vendor raises prices, a benchmark turns out to be flawed), nobody can quickly tell which past decisions rested on it.

1.2 The idea

Why Did We? stores a team's decisions in a FalkorDB graph together with their owners, sources, and the assumptions they depend on. An agent answers questions by walking that graph, takes action when the graph shows impact, remembers changes across sessions, and shows the exact path behind every answer.

1.3 Core capabilities

Capability	What it means in practice
Explain	Trace any decision back to its source, owners, and assumptions
Act	When an assumption changes, find every dependent decision and draft review tickets for the right owners
Remember	Store state changes as graph updates so new sessions start from current state
Coordinate	Two agents (Recorder and Briefing) share state through the graph, not through prompts
Show its work	Every answer includes node IDs and the Cypher that produced it

1.4 Cost model

The whole project runs at zero cost:

Component	Cost
FalkorDB (Docker image)	Free, open source
Docker Desktop	Free for personal use and small teams (check Docker's current terms)
Python, FastAPI, FastMCP, Pydantic, Faker, sentence-transformers	Free, open source
React, Vite, Cytoscape.js	Free, open source
LLM, option A: Ollama	Free. Runs on your own machine.
LLM, option B: Groq	Free tier with rate limits. No paid plan required for the build.

The LLM is behind a single configuration switch, so you can start with Ollama and move to Groq (or back) without touching the agent code.

1.5 Scope

In scope: synthetic project history, text-based ingestion (notes, tickets, chats), a tool-using agent loop that works with either provider, a web UI with a trace panel and graph view, and a scripted demo.

Out of scope: live integrations with Slack, Jira, or Google Drive; authentication; multi-tenant deployment; production ingestion of real company data.

---

2. Choosing an LLM Provider

2.1 Comparison

	Ollama (local)	Groq (hosted)
Cost	Free	Free tier, rate-limited
Data leaves your machine	No	Yes (prompts and tool results are sent to Groq)
Hardware needed	8 GB RAM minimum, 16 GB recommended	Any laptop with internet
Speed	Depends on your hardware; a 7B to 8B model on CPU is slow	Very fast
Tool-calling reliability	Good with qwen2.5:7b and llama3.1:8b; weaker than large hosted models	Good with large Llama models
Internet needed	No, after the model is downloaded	Yes
Limits	Your hardware	Requests per minute and tokens per minute, which vary by model and change over time

2.2 Recommendation

Use Ollama for development and for any demo you need to run offline. Use Groq for the live demo if your laptop is slow or the model is too weak at picking tools. Keep the same code and switch with one environment variable.

2.3 Model choice

Ollama (verify the tag exists with `ollama list` after pulling):

Model	Pull command	Notes
Qwen 2.5 7B	`ollama pull qwen2.5:7b`	Good tool calling at a small size. Default recommendation.
Llama 3.1 8B	`ollama pull llama3.1:8b`	Solid alternative with native tool support.

Groq (model IDs change, so list them before you choose):

```bash
```

```
curl https://api.groq.com/openai/v1/models \
  -H "Authorization: Bearer $LLM_API_KEY"
```

Pick a current Llama 70B-class model that supports tool calling from that list. Larger models select tools more reliably, which matters for the Briefing Agent.

---

3. Architecture

3.1 System diagram

3.2 Components

Component	Responsibility	Technology
Graph store	Single source of truth	FalkorDB, run in Docker
LLM client	One client for both providers	`openai` Python SDK pointed at a base URL
Ingestion	Parse sources, extract records with an LLM, validate	Python, Pydantic
Tool layer	Graph operations as functions with JSON schemas	Python
Tool server	Optional MCP exposure of the same tools	FastMCP (`mcp` package)
Agents	Briefing (questions and actions) and Recorder (writes)	Tool-use loop over chat completions
Backend	REST endpoints and SSE trace stream	FastAPI, uvicorn
Frontend	Chat, trace panel, graph view	React, Vite, TypeScript, Cytoscape.js
Embeddings	Vector index on source text	sentence-transformers (local, free)
Seed data	Synthetic project history	Python, Faker

3.3 Why the OpenAI SDK for both providers

Ollama and Groq both expose an OpenAI-compatible chat completions API that includes tool calling. Using the `openai` SDK with a different `base_url` means the agent loop is written once. The only provider-specific code is the configuration in section 4.

3.4 Design principles

1. The graph is the context. Agents do not carry project state in their prompts.
2. Every fact has provenance. No record enters the graph without a `Source` node.
3. The LLM never writes raw Cypher into the graph. Writes go through typed functions.
4. Tool descriptions are selection rules. The model chooses tools from them.
5. Drafts, not sends. Tickets require human confirmation.
6. The model is a swappable component. Nothing outside `backend/llm/` knows which provider is in use.

---

4. LLM Configuration

4.1 Environment variables

Variable	Required	Example (Ollama)	Example (Groq)
`LLM_PROVIDER`	Yes	`ollama`	`groq`
`LLM_MODEL`	Yes	`qwen2.5:7b`	the model ID you chose in section 2.3
`LLM_API_KEY`	Groq only	(not needed)	your Groq key
`LLM_BASE_URL`	No	defaults to `http://localhost:11434/v1`	defaults to `https://api.groq.com/openai/v1`
`LLM_TEMPERATURE`	No	`0.1`	`0.1`
`LLM_MAX_RETRIES`	No	`4`	`4`

Switching providers means changing `LLM_PROVIDER`, `LLM_MODEL`, and (for Groq) `LLM_API_KEY`. No code changes.

4.2 `backend/config.py`

python

```python
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()

PROVIDER_DEFAULTS = {
    "ollama": {"base_url": "http://localhost:11434/v1", "api_key": "ollama"},
    "groq": {"base_url": "https://api.groq.com/openai/v1", "api_key": None},
}


@dataclass(frozen=True)
class LLMConfig:
    provider: str
    base_url: str
    api_key: str
    model: str
    temperature: float
    max_retries: int


def load_llm_config() -> LLMConfig:
    provider = os.getenv("LLM_PROVIDER", "ollama").strip().lower()
    if provider not in PROVIDER_DEFAULTS:
        raise ValueError(f"LLM_PROVIDER must be one of {list(PROVIDER_DEFAULTS)},

    defaults = PROVIDER_DEFAULTS[provider]
    base_url = os.getenv("LLM_BASE_URL") or defaults["base_url"]
    api_key = os.getenv("LLM_API_KEY") or defaults["api_key"]
    model = os.getenv("LLM_MODEL", "").strip()

    if not model:
        raise ValueError("LLM_MODEL is not set. See the LLM configuration section
    if not api_key:
        raise ValueError(f"LLM_API_KEY is required when LLM_PROVIDER={provider}")

    return LLMConfig(
        provider=provider,
        base_url=base_url,
        api_key=api_key,
        model=model,
        temperature=float(os.getenv("LLM_TEMPERATURE", "0.1")),
        max_retries=int(os.getenv("LLM_MAX_RETRIES", "4")),
    )
```

4.3 `backend/llm/client.py`

python

```python
from functools import lru_cache
from openai import OpenAI
from backend.config import load_llm_config, LLMConfig


@lru_cache(maxsize=1)
def get_config() -> LLMConfig:
    return load_llm_config()


@lru_cache(maxsize=1)
def get_client() -> OpenAI:
    cfg = get_config()
    return OpenAI(base_url=cfg.base_url, api_key=cfg.api_key, timeout=120.0)
```

4.4 `backend/llm/chat.py`

This is the only function the agents call. It retries on rate limits and transient connection errors, and it handles a known Groq failure where the model emits a malformed tool call.

python

```python
import time
from openai import RateLimitError, APIConnectionError, APIStatusError
from backend.llm.client import get_client, get_config


def chat(messages: list[dict], tools: list[dict] | None = None):
    cfg = get_config()
    client = get_client()
    last_error = None

    for attempt in range(cfg.max_retries):
        try:
            kwargs = {
                "model": cfg.model,
                "messages": messages,
                "temperature": cfg.temperature,
            }
            if tools:
                kwargs["tools"] = tools
                kwargs["tool_choice"] = "auto"
            return client.chat.completions.create(**kwargs)

        except RateLimitError as e:
            # Free tiers return 429 when per-minute limits are hit.
            last_error = e
            time.sleep(min(2 ** attempt * 2, 30))

        except APIConnectionError as e:
            # Ollama not running, or network down.
            last_error = e
            time.sleep(2)

        except APIStatusError as e:
            # Groq sometimes returns 400 tool_use_failed when the model writes
            # a malformed tool call. Retrying usually works.
            last_error = e
            if e.status_code == 400 and "tool" in str(e).lower():
                time.sleep(1)
                continue
            raise

    raise RuntimeError(f"LLM call failed after {cfg.max_retries} attempts: {last_e
```

4.5 `.env` examples

Ollama:

```bash
LLM_PROVIDER=ollama
LLM_MODEL=qwen2.5:7b
FALKOR_HOST=localhost
FALKOR_PORT=6379
GRAPH_NAME=whydidwe
```

Groq:

```bash
LLM_PROVIDER=groq
LLM_MODEL=<model id from the Groq models list>
LLM_API_KEY=your-groq-key
FALKOR_HOST=localhost
FALKOR_PORT=6379
GRAPH_NAME=whydidwe
```

---

5. Tool Schemas

Tools use the OpenAI function-calling format, which both providers accept. The schemas live in one file so the docstrings and the schemas stay in sync.

5.1 `backend/tools/schemas.py`

```python
```

```
TOOL_SCHEMAS = [
    {
        "type": "function",
        "function": {
            "name": "cypher_query",
            "description": (
                "Use for read-only lookups that no other tool covers, such as list
                "owners, team members, or decisions by status. Never use for write
                "Returns at most 100 rows."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "A read-only Cypher
                    "params": {"type": "object", "description": "Values referenced
                },
                "required": ["query"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "trace_decision",
            "description": (
                "Use when the user asks why a decision was made or where it came f
                "Walks backward from a decision to its owners, sources, and assump
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "decision_id": {"type": "string"},
                    "max_depth": {"type": "integer", "description": "Default 4"},
                },
                "required": ["decision_id"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "find_dependents",
            "description": (
                "Use when the user asks what breaks, what depends on, or what is a
                "by a changed assumption. Returns active decisions that rely on it
            ),
```

```
            "parameters": {
                "type": "object",
                "properties": {
                    "assumption_id": {"type": "string"},
                    "max_depth": {"type": "integer", "description": "Default 3"},
                },
                "required": ["assumption_id"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "run_centrality",
            "description": (
                "Use when the user asks who the key connectors are or who is most
                "in the network. Returns people ranked by betweenness centrality."
            ),
            "parameters": {
                "type": "object",
                "properties": {"top_n": {"type": "integer", "description": "Defaul
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "draft_ticket",
            "description": (
                "Use after find_dependents, to open a review ticket for a decision
                "Creates a draft that a human must confirm. Never use without a de
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "decision_id": {"type": "string"},
                    "owner_id": {"type": "string"},
                    "severity": {"type": "string", "enum": ["standard", "escalated
                },
                "required": ["decision_id", "owner_id", "severity"],
            },
        },
    },
]

BRIEFING_TOOLS = TOOL_SCHEMAS  # all five tools
```

```
RECORDER_TOOLS = [
    t for t in TOOL_SCHEMAS if t["function"]["name"] in {"cypher_query"}
] + [
    {
        "type": "function",
        "function": {
            "name": "record_decision",
            "description": (
                "Write one decision, its owner, its source, and its assumptions to
                "source_id is required. Call once per decision found in the source
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "id": {"type": "string"},
                    "title": {"type": "string"},
                    "rationale": {"type": "string"},
                    "made_at": {"type": "string", "description": "ISO date"},
                    "owner_id": {"type": "string"},
                    "source_id": {"type": "string"},
                    "assumption_ids": {"type": "array", "items": {"type": "string"
                    "supersedes_id": {"type": "string", "description": "Optional o
                    "reverses_id": {"type": "string", "description": "Optional old
                },
                "required": ["id", "title", "rationale", "made_at", "owner_id", "s
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "update_assumption",
            "description": (
                "Use when a source changes the value of an existing assumption. "
                "Sets the old assumption to invalid and records an Event."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "assumption_id": {"type": "string"},
                    "new_value": {"type": "string"},
                    "source_id": {"type": "string"},
                },
                "required": ["assumption_id", "new_value", "source_id"],
            },
```

```
        },
    },
]
```

---

6. Agent Loop

6.1 `backend/agents/loop.py`

The loop is the same for both agents and both providers. It appends each assistant message and each tool result, then continues until the model answers without a tool call or the iteration cap is reached.

python

```python
import json
from backend.llm.chat import chat

MAX_ITERATIONS = 8
MAX_TOOL_RESULT_CHARS = 8000


def run_agent(system_prompt: str, user_message: str, tools: list[dict],
              dispatch, emit) -> str:
    """
    dispatch(name, args) -> JSON-serializable result. Raises on failure.
    emit(event: dict) -> None. Sends trace and token events to the API layer.
    """
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_message},
    ]

    for step in range(1, MAX_ITERATIONS + 1):
        response = chat(messages, tools=tools)
        msg = response.choices[0].message

        assistant_entry = {"role": "assistant", "content": msg.content or ""}
        if msg.tool_calls:
            assistant_entry["tool_calls"] = [
                {
                    "id": tc.id,
                    "type": "function",
                    "function": {"name": tc.function.name, "arguments": tc.functio
                }
                for tc in msg.tool_calls
            ]
        messages.append(assistant_entry)

        if not msg.tool_calls:
            emit({"type": "token", "text": msg.content or ""})
            emit({"type": "done"})
            return msg.content or ""

        for tc in msg.tool_calls:
            name = tc.function.name
            try:
                args = json.loads(tc.function.arguments or "{}")
                result = dispatch(name, args)
                ok = True
            except json.JSONDecodeError:
```

```
                args = {"_raw": tc.function.arguments}
                result = {"error": "Tool arguments were not valid JSON. Retry with
                ok = False
            except Exception as e:
                args = args if "args" in locals() else {}
                result = {"error": f"{type(e).__name__}: {e}"}
                ok = False

            emit({
                "type": "trace",
                "step": step,
                "tool": name,
                "input": args,
                "ok": ok,
                "result_summary": _summarize(result),
            })

            messages.append({
                "role": "tool",
                "tool_call_id": tc.id,
                "content": json.dumps(result, default=str)[:MAX_TOOL_RESULT_CHARS]
            })

    emit({"type": "done", "note": "iteration limit reached"})
    return ""


def _summarize(result) -> str:
    if isinstance(result, dict) and "error" in result:
        return f"error: {result['error']}"
    if isinstance(result, list):
        return f"{len(result)} rows"
    return "ok"
```

6.2 Notes on the loop

Every tool call is logged as a trace event, which feeds the UI trace panel. Tool results are truncated to 8,000 characters so small local models with limited context windows do not overflow. If a model returns malformed arguments, the loop returns an error message to the model rather than crashing, which gives it a chance to correct itself.

---

7. Briefing Agent

7.1 `backend/agents/briefing.py`

python

```python
from backend.agents.loop import run_agent
from backend.tools.dispatch import dispatch
from backend.tools.schemas import BRIEFING_TOOLS

SYSTEM_PROMPT = """You answer questions about a team's decisions using a graph dat
You have tools. Choose them by the rules below, and never answer from memory when

Tool selection:
- "Why did we...?" or "where did X come from?" -> trace_decision
- "What breaks if X changes?" or "what depends on X?" -> find_dependents
- "Who is the key connector?" or "who is most central?" -> run_centrality
- Owner or membership questions -> cypher_query (read-only)
- To open a review after find_dependents -> draft_ticket, one call per owner

Rules:
- Cite node IDs for every decision, person, and source you mention.
- Never invent IDs. If a tool returns nothing, say so.
- When a changed assumption affects several decisions, rank them by depth and say
- Keep answers short. Lead with the answer, then the path.
"""


def answer(question: str, emit) -> str:
    return run_agent(
        system_prompt=SYSTEM_PROMPT,
        user_message=question,
        tools=BRIEFING_TOOLS,
        dispatch=dispatch,
        emit=emit,
    )
```

7.2 Tool selection rules

Question type	Tool
"Why did we...?"	`trace_decision`
"What breaks if X changes?"	`find_dependents`
"Who is the key connector?"	`run_centrality`
"Who owns or knows X?"	`cypher_query`
"Open a review for X"	`draft_ticket`, after `find_dependents`
Anything else	`cypher_query`

Smaller local models are more likely to pick the wrong tool for ambiguous questions. The phase 3 checkpoint in section 14 tests this directly. If a model fails a case, add one worked example to the system prompt before switching models.

---

8. Recorder Agent

8.1 Extraction prompt and parsing

Smaller models do not always return clean JSON, and Groq's JSON mode and Ollama's format options differ. This design does not depend on either. It asks for JSON, strips code fences, parses, and validates with Pydantic, with one retry that includes the validation error.

`backend/ingest/models.py`:

```python
from pydantic import BaseModel, Field


class ExtractedPerson(BaseModel):
    id: str
    name: str
    role: str = ""


class ExtractedAssumption(BaseModel):
    id: str
    claim: str
    value: str


class ExtractedDecision(BaseModel):
    id: str
    title: str
    rationale: str
    made_at: str
    owner_id: str
    assumption_ids: list[str] = Field(default_factory=list)
    supersedes_id: str | None = None
    reverses_id: str | None = None


class ExtractionResult(BaseModel):
    people: list[ExtractedPerson] = Field(default_factory=list)
    assumptions: list[ExtractedAssumption] = Field(default_factory=list)
    decisions: list[ExtractedDecision] = Field(default_factory=list)
```

`backend/ingest/extract.py`:

python

```python
import json
import re
from pydantic import ValidationError
from backend.llm.chat import chat
from backend.ingest.models import ExtractionResult

EXTRACT_PROMPT = """Extract decisions from the source text below.

Return ONLY a JSON object with this shape, with no preamble and no code fences:
{
  "people": [{"id": "...", "name": "...", "role": "..."}],
  "assumptions": [{"id": "...", "claim": "...", "value": "..."}],
  "decisions": [{"id": "...", "title": "...", "rationale": "...", "made_at": "YYYY
                 "owner_id": "...", "assumption_ids": ["..."],
                 "supersedes_id": null, "reverses_id": null}]
}

Rules:
- Use short lowercase IDs with hyphens (e.g. "adopt-postgres").
- Only include decisions explicitly made in the text.
- owner_id must match a person id in "people".
- Use empty lists when nothing applies.

SOURCE:
"""


def _parse(raw: str) -> dict:
    cleaned = re.sub(r"^```(?:json)?\s*|\s*```$", "", raw.strip())
    start, end = cleaned.find("{"), cleaned.rfind("}")
    if start == -1 or end == -1:
        raise ValueError("No JSON object found in model output")
    return json.loads(cleaned[start:end + 1])


def extract(source_text: str) -> ExtractionResult:
    messages = [{"role": "user", "content": EXTRACT_PROMPT + source_text}]

    for attempt in range(2):
        reply = chat(messages).choices[0].message.content or ""
        try:
            return ExtractionResult.model_validate(_parse(reply))
        except (ValueError, json.JSONDecodeError, ValidationError) as e:
            if attempt == 1:
                raise
            messages.append({"role": "assistant", "content": reply})
```

```
            messages.append({"role": "user", "content":
                f"That output failed validation: {e}. Return only the corrected JS

    raise RuntimeError("unreachable")
```

8.2 Recorder loop

The Recorder runs the same loop as the Briefing Agent, but with write tools and a prompt that states the write rules. It receives a source already extracted and validated, or it extracts and then calls `record_decision` for each item.

`backend/agents/recorder.py`:

python

```python
from backend.agents.loop import run_agent
from backend.tools.dispatch import dispatch
from backend.tools.schemas import RECORDER_TOOLS

SYSTEM_PROMPT = """You record decisions from a new source into a graph.

For each decision in the source:
1. Call cypher_query to check whether a decision on the same topic already exists.
2. If it exists and the new source reverses it, pass reverses_id.
3. If it exists and the new source replaces it, pass supersedes_id.
4. Call record_decision with the source_id you were given. Never omit it.

For each assumption whose value changed, call update_assumption.
Do not invent people or IDs. Stop when every decision and change is recorded.
"""


def record(source_id: str, extracted_json: str, emit) -> str:
    user_message = f"source_id: {source_id}\nExtracted records:\n{extracted_json}"
    return run_agent(
        system_prompt=SYSTEM_PROMPT,
        user_message=user_message,
        tools=RECORDER_TOOLS,
        dispatch=dispatch,
        emit=emit,
    )
```

8.3 Conflict cases

Case	Graph change
New value for an existing assumption	`update_assumption` sets the old one to `valid=false`, creates the new value, writes an `Event` with `kind='assumption_changed'`
Decision reversed	`record_decision` with `reverses_id`: new decision, `REVERSES` edge, old status set to `reversed`, `Event` written
Decision replaced	`record_decision` with `supersedes_id`: `SUPERSEDES` edge, old status set to `superseded`

Ingestion checks that the model did not invent people. Any `owner_id` not found in the graph or in the extraction's `people` list is rejected before the write.

---

9. Tool Layer

9.1 Tool list

Tool	Access	Purpose
`cypher_query`	Read-only	Ad hoc lookups
`trace_decision`	Read-only	Backward walk from a decision
`find_dependents`	Read-only	Blast radius of an assumption
`run_centrality`	Read-only	Key connectors
`record_decision`	Write	Recorder only
`update_assumption`	Write	Recorder only
`draft_ticket`	Write (draft)	Briefing Agent, after confirmation rules

Write tools are available only to the Recorder (`RECORDER_TOOLS`) except `draft_ticket`, which the Briefing Agent uses to create drafts only.

9.2 Dispatch

`backend/tools/dispatch.py` maps tool names to functions and rejects anything not on the list. The model cannot call a function it was not given, and the dispatcher adds a second check.

```python
```

```python
from backend.tools.cypher import cypher_query
from backend.tools.traversal import trace_decision, find_dependents
from backend.tools.algorithms import run_centrality
from backend.tools.writes import record_decision, update_assumption, draft_ticket

REGISTRY = {
    "cypher_query": cypher_query,
    "trace_decision": trace_decision,
    "find_dependents": find_dependents,
    "run_centrality": run_centrality,
    "record_decision": record_decision,
    "update_assumption": update_assumption,
    "draft_ticket": draft_ticket,
}


def dispatch(name: str, args: dict):
    if name not in REGISTRY:
        raise ValueError(f"Unknown tool: {name}")
    return REGISTRY[name](**args)
```

9.3 Read-only guard

`backend/tools/cypher.py`:

python

```python
import re
from backend.graph.client import get_graph

WRITE_PATTERN = re.compile(
    r"\b(CREATE|MERGE|DELETE|DETACH|SET|REMOVE|DROP|CALL\s+db\.|LOAD\s+CSV)\b",
    re.IGNORECASE,
)
MAX_ROWS = 100


def cypher_query(query: str, params: dict | None = None):
    if WRITE_PATTERN.search(query):
        return {"error": "Write clauses are not allowed in cypher_query."}
    g = get_graph()
    result = g.ro_query(query, params or {})  # read-only path
    rows = result.result_set[:MAX_ROWS]
    return [dict(zip(result.header, row)) for row in rows] if result.header else r
```

Check that `ro_query` exists in your installed `falkordb` client version. If it does not, use `query` and rely on the keyword guard plus parameter binding. The guard tests in `backend/tests/test_guard.py` cover both paths.

9.4 Traversal and algorithms

`find_dependents` and `trace_decision` use the Cypher patterns in section 10.3. `run_centrality` calls FalkorDB's betweenness procedure. Store its exact name and arguments in `backend/graph/schema.py` after checking `CALL dbms.procedures()` in the FalkorDB Browser.

9.5 MCP exposure (optional)

The same functions can be published through FastMCP so a separate MCP client can call them. This is useful for showing the tool-selection layer, but the agent loop does not depend on it.

```python
from mcp.server.fastmcp import FastMCP
from backend.tools.dispatch import REGISTRY

mcp = FastMCP("whydidwe")
for _name, _fn in REGISTRY.items():
    mcp.tool(name=_name)(_fn)

if __name__ == "__main__":
    mcp.run()
```

---

10. Graph

10.1 Node labels

Label	Properties	Purpose
`Decision`	`id`, `title`, `status` (active, reversed, superseded), `made_at`, `rationale`, `track`	Core hub
`Person`	`id`, `name`, `role`	People who decide or own work
`Team`	`id`, `name`	Groups
`Source`	`id`, `type` (meeting, ticket, chat, doc), `uri`, `date`, `text`, `embedding`	Provenance
`Assumption`	`id`, `claim`, `value`, `valid`, `updated_at`	Facts decisions rest on
`Event`	`id`, `kind` (decided, reversed, superseded, assumption_changed), `at`	Episodic memory
`Pattern`	`id`, `rule`, `source_count`	Procedural memory
`Ticket`	`id`, `title`, `status` (draft, confirmed, closed), `owner`, `severity`	Action output

10.2 Edge types

Edge	From → To	Meaning
`DECIDED_BY`	Decision → Person	Who made the call
`SOURCED_FROM`	Decision → Source	Where it was discussed
`DEPENDS_ON`	Decision → Assumption, or Decision → Decision	What it rests on
`REVERSES`	Decision → Decision	Undid an earlier decision
`SUPERSEDES`	Decision → Decision	Replaced an earlier decision
`OWNS`	Person or Team → Decision or Ticket	Ownership
`MEMBER_OF`	Person → Team	Membership
`TRIGGERED`	Event → Decision or Assumption	Episodic link
`APPLIES_TO`	Pattern → Decision	Procedural rule
`RAISED`	Ticket → Decision	Ticket opened about a decision

10.3 Key Cypher patterns

Trace a decision backward:

```cypher
MATCH path = (d:Decision {id: $id})-[:DECIDED_BY|SOURCED_FROM|DEPENDS_ON*1..4]->(n
RETURN path
```

Find active dependents of an assumption:

```cypher
MATCH (a:Assumption {id: $id})<-[r:DEPENDS_ON*1..3]-(d:Decision {status: 'active'}
OPTIONAL MATCH (d)-[:DECIDED_BY]->(p:Person)
RETURN d.id AS decision, d.title AS title, p.id AS owner_id, p.name AS owner,
       min(length(r)) AS depth
ORDER BY depth ASC
```

Shortest path between two people:

```cypher
```

```
MATCH (a:Person {id: $from}), (b:Person {id: $to}),
      p = shortestPath((a)-[:MEMBER_OF|OWNS|DECIDED_BY*..6]-(b))
RETURN p
```

Centrality: call the betweenness procedure your FalkorDB version provides. Confirm the name and arguments first.

10.4 Schema setup

`backend/graph/schema.py` creates uniqueness constraints on `id` for each label, an index on `Decision.status` and `Assumption.valid`, and the vector index on `Source.embedding` using the syntax in the FalkorDB documentation for your version (dimension 384 for `all-MiniLM-L6-v2`).

---

11. Ingestion Pipeline

1. Read the source file (markdown, text, or CSV).
2. Create a `Source` node with `id`, `type`, `uri`, `date`, and `text`.
3. Compute an embedding with sentence-transformers and store it on the node.
4. Call `extract()` (section 8.1) to get validated records.
5. Reject any `owner_id` that does not resolve to a person in the graph or the extraction.
6. Write people, assumptions, and decisions with `UNWIND` batches.
7. Run the Recorder for conflict handling and state changes (section 8.2).
8. Write one `Event` node per state change.

11.1 Batch loader example

cypher

```
UNWIND $rows AS r
MERGE (d:Decision {id: r.id})
SET d.title = r.title, d.status = 'active',
    d.made_at = r.made_at, d.rationale = r.rationale
```

Load order: Teams, People, Sources, Assumptions, Decisions, then edges.

11.2 Seed data

Generated by `data/seed/generate.py` (Faker, no LLM needed):

Item	Target
Decisions	50 to 100
People	15 to 20
Teams	4
Sources	150 to 250
Assumptions	about 80
Planted dependency clusters	2 or 3, one assumption feeding 3 to 6 decisions
Reversed or superseded decisions	about 10, including a repeated pattern

Because the seed data is generated without an LLM, the build needs no model calls to load the graph. Use the LLM only for the Recorder's live ingestion step.

11.3 Caching extraction results

Model calls on small local models are slow, and free-tier rate limits are finite. Save every extraction result as JSON in `data/seed/extracted/<source_id>.json`. The loader reads from that cache when it exists, so you extract each source at most once.

---

12. API

Method	Path	Purpose	Response
POST	`/chat`	Ask the Briefing Agent	SSE: `trace`, `token`, `done`
POST	`/ingest`	Run extraction and the Recorder on a source	JSON summary
GET	`/graph/subgraph`	Neighborhood for the graph view	`{nodes, edges}`; query params `node`, `depth`
GET	`/tickets`	List tickets	JSON list
POST	`/tickets/{id}/confirm`	Confirm a draft ticket	Updated ticket
GET	`/health`	Reports the graph and LLM provider status	`{graph: ok, llm: provider/model}`

`backend/api/main.py` streams the trace events from `run_agent` as server-sent events. The `/health` endpoint is useful during the demo because it confirms which provider and model are active.

---

13. Frontend

Panel	Contents
Left: chat	Input, streamed answer, ticket cards with Confirm
Center: trace	One row per step: tool, input, `ok`, result summary
Right: graph	Cytoscape subgraph around the most recently referenced node

Clicking a trace step highlights its node IDs. The header shows the active provider and model from `/health`, so judges can see the setup. Keep styling minimal.

Stack: React, Vite, TypeScript, `cytoscape`, `react-cytoscapejs`, `react-markdown`.

---

14. Installation

14.1 Prerequisites

Tool	Version	Check
Docker Desktop	Current	`docker --version`
Python	3.11 or newer	`python3 --version`
Node.js	20 or newer	`node --version`
Git	Any recent	`git --version`
Ollama (option A)	Current	`ollama --version`
Groq API key (option B)	Free account	console.groq.com

You need Ollama, a Groq key, or both. Ollama is the zero-cost default.

14.2 Install Docker Desktop

1. Download from https://www.docker.com/products/docker-desktop/.
2. Run the installer. On Windows, enable WSL 2 when prompted.
3. Start Docker Desktop and wait until it reports it is running.
4. Verify:

```bash
docker --version
docker run hello-world
```

14.3 Run FalkorDB

```bash
docker run -d --name falkordb \
  -p 6379:6379 \
  -p 3000:3000 \
  -v falkordb_data:/var/lib/falkordb/data \
  falkordb/falkordb:latest
```

Verify:

```bash
docker exec -it falkordb redis-cli PING     # expected: PONG
```

Open http://localhost:3000 to confirm the FalkorDB Browser loads.

Lifecycle commands:

```bash
docker stop falkordb
docker start falkordb
docker logs -f falkordb
docker rm -f falkordb                # container removed, data kept in volume
docker volume rm falkordb_data       # wipe data
```

Run `CALL dbms.procedures()` in the Browser and record the exact algorithm procedure names for your version.

14.4 Option A: Install Ollama (free, local)

1. Download the installer from https://ollama.com/download and install it.
2. Pull a tool-capable model. The 7B model needs about 8 GB of RAM free:

```bash
ollama pull qwen2.5:7b
```

3. Make sure the server is running. On most installs it starts automatically. If not:

```bash
ollama serve
```

4. Verify the OpenAI-compatible endpoint:

```bash
```

```
curl http://localhost:11434/v1/models
```

The response lists your pulled models. If `qwen2.5:7b` is missing, the pull failed.

5. Test a chat request:

```bash
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"qwen2.5:7b","messages":[{"role":"user","content":"Reply with OK"}]
```

If your machine has 16 GB or more, `llama3.1:8b` is a good alternative. On 8 GB machines, expect slow responses; keep the seed set small.

14.5 Option B: Get a Groq API key (free tier)

1. Create an account at https://console.groq.com.
2. Create an API key in the console and copy it once.
3. Check the free-tier limits shown in the console. They vary by model and can change.
4. List models available to your key:

```bash
curl https://api.groq.com/openai/v1/models \
  -H "Authorization: Bearer YOUR_KEY"
```

5. Choose a current model from that list that supports tool calling, and put its ID in `LLM_MODEL`.

Groq receives your prompts and tool results. Do not load real confidential data into this project when using Groq.

14.6 Project setup

```bash
mkdir why-did-we && cd why-did-we
git init
mkdir -p backend/{llm,ingest,graph,tools,agents,api,tests} data/seed/extracted fro
python3 -m venv .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.ps
```

`.gitignore`:

```
.venv/
__pycache__/
.env
node_modules/
frontend/dist/
data/seed/extracted/
```

14.7 Python dependencies

bash

```
pip install --upgrade pip
pip install falkordb openai mcp fastapi "uvicorn[standard]" pydantic \
  python-dotenv sentence-transformers networkx faker pytest
pip freeze > backend/requirements.txt
```

Package	Purpose
`falkordb`	FalkorDB client
`openai`	Client for Ollama and Groq (OpenAI-compatible API)
`mcp`	Optional FastMCP exposure
`fastapi`, `uvicorn`	Backend and SSE
`pydantic`	Validation of extracted records
`python-dotenv`	Load `.env`
`sentence-transformers`	Local embeddings
`networkx`	Test oracle
`faker`	Seed data
`pytest`	Tests

Note: `sentence-transformers` installs PyTorch, which is large (around 1 to 2 GB). Install it on a machine with enough disk space.

14.8 Frontend setup

bash

```
cd frontend
npm create vite@latest . -- --template react-ts
npm install
npm install cytoscape react-cytoscapejs react-markdown
cd ..
```

Run with `npm run dev` from `frontend/`.

14.9 Environment file

Create `.env` at the project root. Use the block for your provider (section 4.5). Never commit `.env`.

14.10 Verify the stack

Create `backend/check_setup.py`:

```python
```

```python
from dotenv import load_dotenv
from falkordb import FalkorDB
import os

from backend.llm.client import get_client, get_config

load_dotenv()

# 1. Graph
db = FalkorDB(host=os.getenv("FALKOR_HOST", "localhost"),
              port=int(os.getenv("FALKOR_PORT", 6379)))
g = db.select_graph(os.getenv("GRAPH_NAME", "whydidwe"))
g.query("CREATE (:Health {ok: true})")
print("FalkorDB row:", g.query("MATCH (n:Health) RETURN n.ok").result_set)
g.query("MATCH (n:Health) DELETE n")

# 2. LLM plain reply
cfg = get_config()
client = get_client()
reply = client.chat.completions.create(
    model=cfg.model,
    messages=[{"role": "user", "content": "Reply with OK"}],
    max_tokens=10,
)
print(f"LLM ({cfg.provider}/{cfg.model}) reply:", reply.choices[0].message.content

# 3. LLM tool calling (this is the check that matters most)
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get weather for a city",
        "parameters": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
        },
    },
}]
resp = client.chat.completions.create(
    model=cfg.model,
    messages=[{"role": "user", "content": "What is the weather in Pune? Use the to
    tools=tools,
    tool_choice="auto",
)
```

```
calls = resp.choices[0].message.tool_calls or []
print("Tool call:", [(c.function.name, c.function.arguments) for c in calls] or "N
```

Run from the project root:

```bash
python -m backend.check_setup
```

Expected: a FalkorDB row, an OK reply, and a `get_weather` tool call with a city argument. If the tool call line says NONE, the model does not support tool calling through this endpoint. Switch models before continuing. Qwen 2.5 and Llama 3.1 both support it through Ollama and Groq.

---

15. Repository Layout

```
why-did-we/
├── .env
├── .gitignore
├── README.md
├── .venv/
├── backend/
│   ├── requirements.txt
│   ├── config.py                # provider configuration
│   ├── check_setup.py           # section 14.10
│   ├── llm/
│   │   ├── client.py            # OpenAI-compatible client, cached
│   │   └── chat.py              # chat() with retries
│   ├── graph/
│   │   ├── client.py            # FalkorDB connection
│   │   ├── loader.py            # UNWIND writers
│   │   └── schema.py            # constraints, indexes, procedure names
│   ├── ingest/
│   │   ├── parse.py             # read files, create Source nodes
│   │   ├── models.py            # Pydantic records
│   │   ├── extract.py           # extraction prompt and parsing
│   │   └── conflicts.py         # conflict helpers
│   ├── tools/
│   │   ├── schemas.py           # OpenAI-format tool schemas
│   │   ├── dispatch.py          # name → function registry
│   │   ├── cypher.py            # read-only cypher_query
│   │   ├── traversal.py         # trace_decision, find_dependents
│   │   ├── algorithms.py        # run_centrality
│   │   ├── writes.py            # record_decision, update_assumption,
draft_ticket
│   │   └── server.py            # optional FastMCP exposure
│   ├── agents/
│   │   ├── loop.py              # shared tool-use loop
│   │   ├── briefing.py          # Briefing Agent
│   │   └── recorder.py          # Recorder Agent
│   ├── api/
│   │   └── main.py              # FastAPI, SSE
│   └── tests/
│       ├── test_guard.py
│       ├── test_tools.py
│       └── test_extract.py
├── data/
│   └── seed/
│       ├── generate.py
│       ├── sources/             # generated notes, tickets, chats
```

```
│       ├── extracted/            # cached extraction JSON (gitignored)
│       └── pricing_note.md       # live ingestion demo input
├── frontend/
│   └── src/
│       ├── App.tsx
│       ├── api.ts
│       └── components/
│           ├── Chat.tsx
│           ├── TracePanel.tsx
│           ├── GraphView.tsx
│           └── TicketCard.tsx
└── docs/
    ├── architecture.md
    └── demo-script.md
```

Run commands from the project root, so imports like `backend.config` resolve. Add an empty `backend/__init__.py` and subpackage `__init__.py` files if your Python version needs them.

---

16. Build Plan

Estimates assume one builder full time. Compress Phases 5 to 7 first if time runs short.

Phase 0: Setup (half a day)

Complete section 14. Pass `check_setup.py` including the tool-call line. Record the chosen provider and model in `README.md`.

Exit criteria: graph row, LLM reply, and tool call all succeed.

Phase 1: Data and graph (1 day)

Write `data/seed/generate.py` (no LLM required). Write the loader with `UNWIND` batches. Create constraints and indexes. Build the vector index using the FalkorDB syntax for your version.

Checkpoint: run five hand-written Cypher queries in the Browser: trace one decision, list dependents of one assumption, count decisions per team, shortest path between two people, list reversed decisions.

Phase 2: Tool layer (1 day)

Implement the functions in section 9. Write `schemas.py` and `dispatch.py`. Write the guard tests.

Checkpoint: `pytest backend/tests/test_guard.py` passes. Each tool returns the expected result on the planted cluster when called directly.

Phase 3: Briefing Agent and provider switch (1 day)

Implement `loop.py` and `briefing.py`. Run the four-question checkpoint:

Question	Expected tool
Why did we switch to Postgres?	`trace_decision`
What depends on the Q1 latency benchmark?	`find_dependents`
Who is the key connector on the platform team?	`run_centrality`
Who owns the caching decision?	`cypher_query`

Run the checkpoint once with Ollama and once with Groq by changing `LLM_PROVIDER` only. Record which passes. If the model picks the wrong tool, add one worked example to the system prompt. Do not change the tool descriptions to fix a single case.

Phase 4: Recorder and ingestion (1 day)

Implement `extract.py` with the parse and retry logic. Implement the Recorder loop. Run extraction on the seed sources and cache the results.

Checkpoint: ingest `pricing_note.md`. The assumption flips to `valid=false`, an `Event` exists, and dependent decisions are still queryable. Run the extraction tests in `test_extract.py` with a malformed-JSON fixture.

Phase 5: API (0.5 day)

Build the endpoints in section 12. Include `/health`. Test with curl. The `/chat` endpoint should emit trace events before tokens.

Phase 6: Frontend (1 day)

Build the panels in section 13. Show the provider and model from `/health` in the header.

Checkpoint: the full demo runs in the browser.

Phase 7: Demo hardening (0.5 day)

Run the demo five times on the chosen provider. Keep a cached extraction for the pricing note so the live step does not depend on a model call. Record a backup video. Finish `README.md`.

If the live model is too slow on stage, pre-record the agent's answers and run the trace from the cache, and say so in the demo.

---

17. Demo Script (Track 01)

1. Ask "why did we switch to Postgres?" The trace shows the decision, the source, the owners, and the benchmark.
2. Ingest `pricing_note.md`, which says the benchmark used a flawed dataset. The assumption flips to invalid and the graph highlights it.
3. The Briefing Agent runs `find_dependents`, ranks three active decisions by depth, and drafts tickets. The closest decision gets an escalated ticket.

4. Confirm one ticket and show its dependency path.
5. Open a new chat and ask a follow-up. The answer uses the updated state.

Point out the provider line in the header. Saying "this runs on a free local model" is a strength, not a limitation, since it means no data leaves the machine and no account is needed.

For Track 02, spend more time on steps 2 and 5 and show the Recorder and Briefing agents on the same graph. For Track 03, start with people and teams, trace a decision to its documents, then ingest a newer document and show the old decision marked superseded.

---

18. Testing

Level	What	Tool
Guard	Write keywords rejected; parameters bound	pytest, `test_guard.py`
Tools	Expected results on seed data	pytest, networkx as oracle
Extraction	Malformed JSON recovered or rejected; code fences stripped	pytest, `test_extract.py`
Agent	Tool selection for the four question types, on each provider	Manual checklist, phase 3
Provider switch	Same run with `LLM_PROVIDER` changed	Manual
End to end	Demo script	Manual, five runs

Use networkx on an export of the seed graph to verify shortest paths and betweenness results against FalkorDB.

---

19. Risks and Mitigations

Risk	Mitigation
Small local model picks the wrong tool	Test in phase 3; add one worked example; use a larger model if available
Model returns malformed JSON	Fence stripping, one retry with the error, Pydantic validation, cached extraction
Groq rate limits interrupt a run	Retry with backoff in `chat()`; cache extraction; keep the demo to a few calls
Groq returns malformed tool calls	Retry on 400 in `chat()`; the schemas keep arguments simple
Ollama is slow on the demo machine	Pre-cache extraction; pre-record a backup run; use Groq for the live demo if needed
Data privacy concerns with Groq	Use only synthetic data, which this project does by design
Procedure names differ across FalkorDB versions	Verify in Phase 0; store names as constants
Model identifiers change	Check `/models` (Groq) or `ollama list` before the build starts
Running out of time	Cut order below

Cut order: vector index, optional MCP exposure, graph view animation, Track 02 and 03 extras. Never cut the action flow, `find_dependents`, the trace panel, the provider switch, or the fresh-session memory demo.

---

20. Decision Log

Decision	Choice	Reason
Primary track	Track 01	The action loop is the easiest to show on stage
Graph store	FalkorDB	Required; Cypher and built-in algorithms
LLM access	OpenAI-compatible client with Ollama or Groq	Zero cost; provider can change with configuration
Default provider	Ollama	No account, no data leaves the machine, no rate limits
Hosted option	Groq	Faster responses for a live demo, free tier
Tool format	OpenAI function-calling schema	Accepted by both providers
Tool exposure	Python registry; FastMCP optional	Keeps the agent loop independent of MCP
Ingestion	Prompted JSON with Pydantic validation and one retry	Works with models that have no JSON mode
Embeddings	Local sentence-transformers	Free, offline
Graph UI	Cytoscape.js	Good path highlighting
Ticket behavior	Draft, then human confirm	Avoids acting on real owners by accident

21. Glossary

Term	Meaning
Blast radius	Decisions affected by a changed assumption
Dependent	A decision that reaches an assumption through `DEPENDS_ON`
Episodic memory	Dated records of what happened
Procedural memory	Learned rules that guide actions
Semantic memory	Current state of decisions and assumptions
Provenance	The source a fact came from
Tool calling	The model returns a function name and arguments; your code runs it and returns the result
OpenAI-compatible API	A chat API shape used by Ollama, Groq, and others, so one client works with all of them
SSE	Server-sent events, the stream format for trace updates
MCP	Model Context Protocol, a standard for exposing tools to agents

---

22. Open Items

Confirm before building: the Groq model ID that supports tool calling on your account, the Ollama model you pulled and its RAM use on your machine, the exact FalkorDB procedure names for your version, and the vector index syntax. Each is noted in the section where it applies.